



Accelerating DiT Inference with Skip Softmax Attention

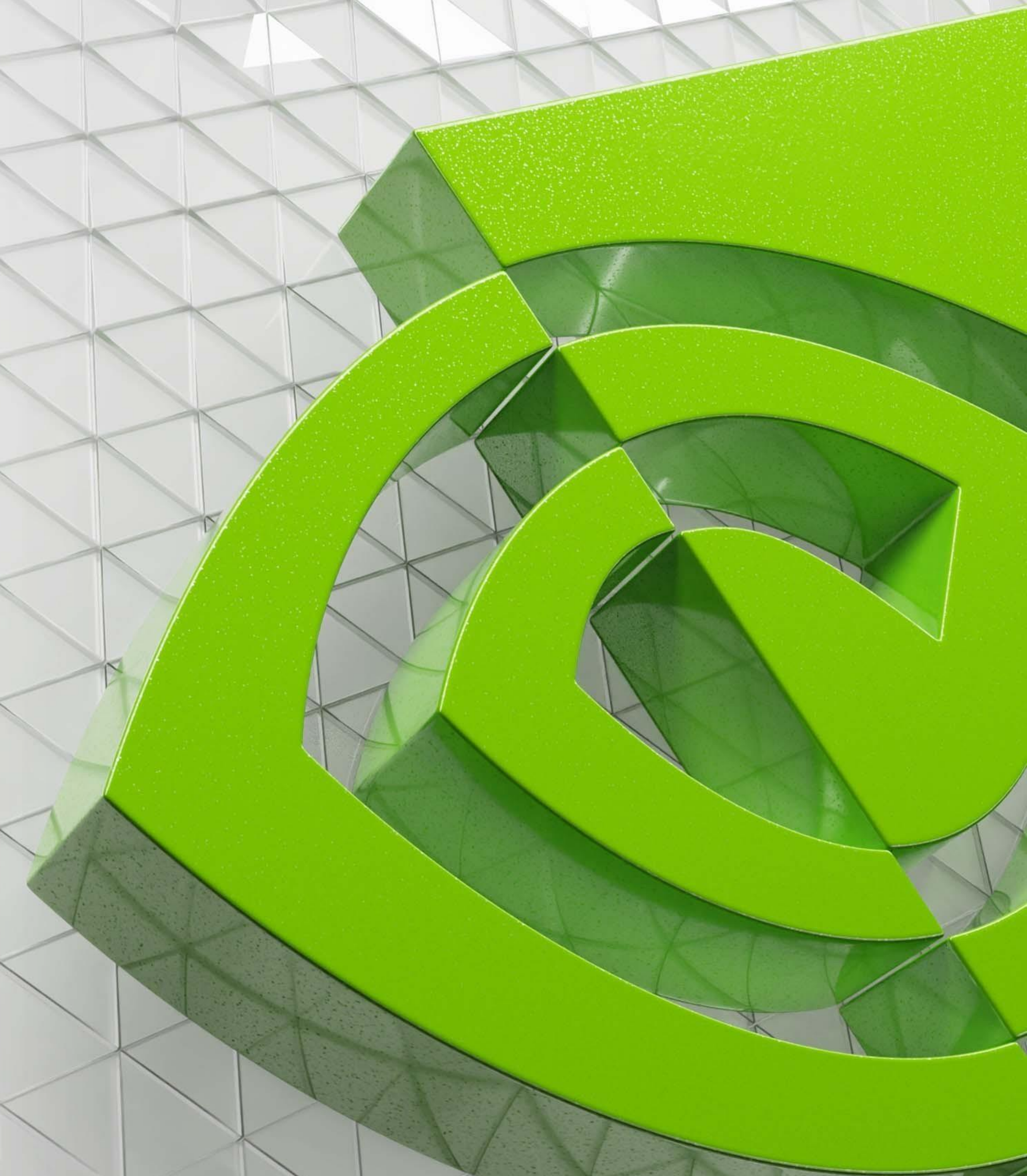
Bo Li, DevTech Compute



Agenda

Method Introduction

Experiment Results



Algorithm Overview

Algorithm 1 FlashAttention with BLASST

Require: Query blocks $\{Q_i\}_{i=1}^{T_r}$, Key blocks $\{K_j\}_{j=1}^{T_c}$,
Value blocks $\{V_j\}_{j=1}^{T_c}$, threshold λ

Ensure: Output blocks $\{O_i\}_{i=1}^{T_r}$

```

1: for  $i = 1$  to  $T_r$  do
2:   Initialize  $m_i^{(0)} = -\infty, O_i^{(0)} = 0, l_i^{(0)} = 0$ 
3:   for  $j = 1$  to  $T_c$  do
4:     Compute  $S_{ij} = Q_i K_j^\top$   $\triangleright$  Attention scores
5:      $\tilde{m}_i^{(j)} = \text{rowmax}(S_{ij})$   $\triangleright$  Local maximum
6:      $m_i^{(j)} = \max(m_i^{(j-1)}, \tilde{m}_i^{(j)})$   $\triangleright$  Running maximum
7:     if  $\tilde{m}_i^{(j)} - m_i^{(j)} < \ln(\lambda)$  then
8:       continue  $\triangleright$  Skip this block
9:     end if
10:     $\tilde{P}_{ij} = \exp(S_{ij} - m_i^{(j)})$   $\triangleright$  Compute attn. weights
11:     $l_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} l_i^{(j-1)} + \text{rowsum}(\tilde{P}_{ij})$ 
12:     $O_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} O_i^{(j-1)} + \tilde{P}_{ij} V_j$ 
13:   end for
14:    $O_i = O_i^{(T_c)} / l_i^{(T_c)}$   $\triangleright$  Final normalization
15: end for
16: return  $\{O_i\}_{i=1}^{T_r}$ 

```

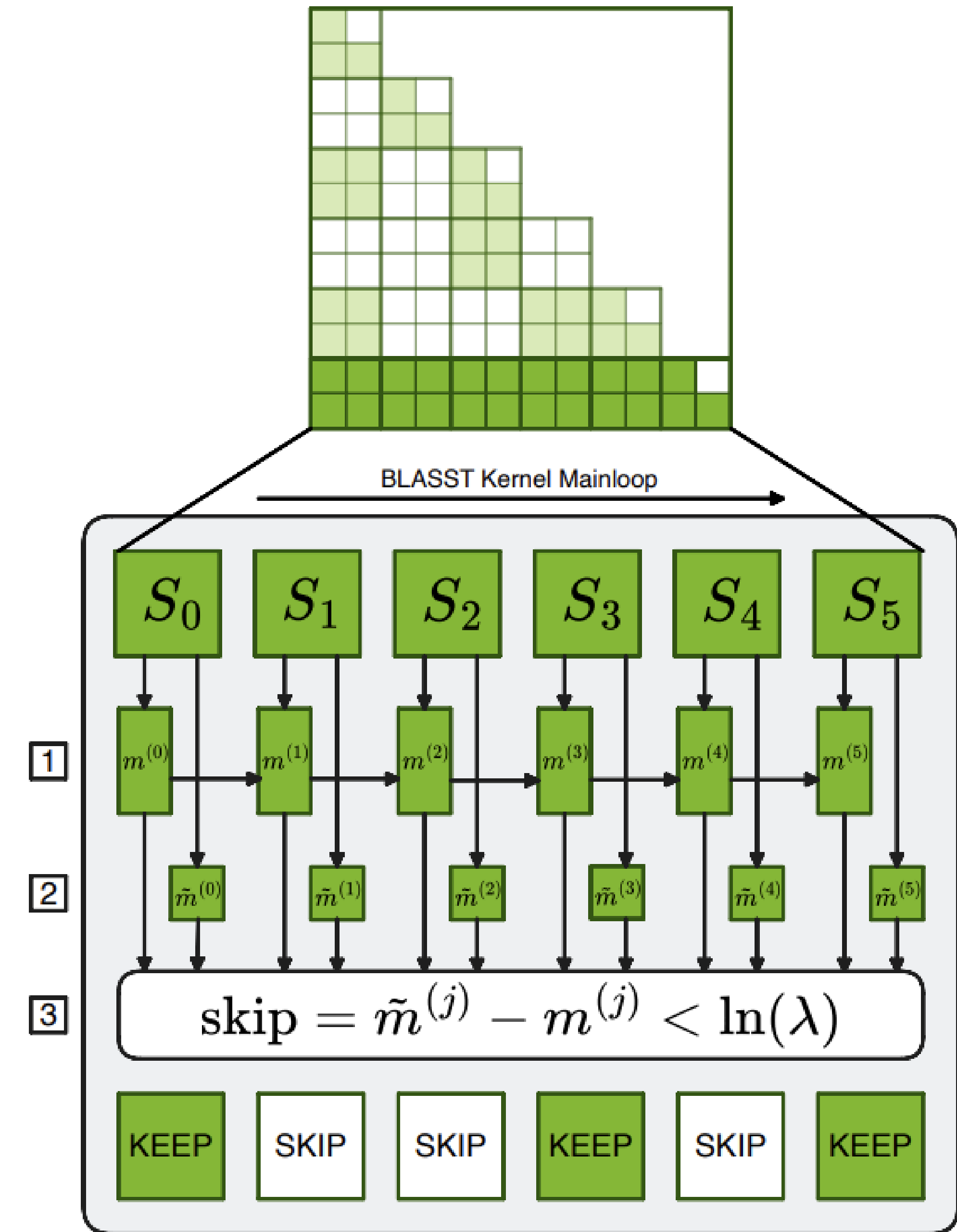
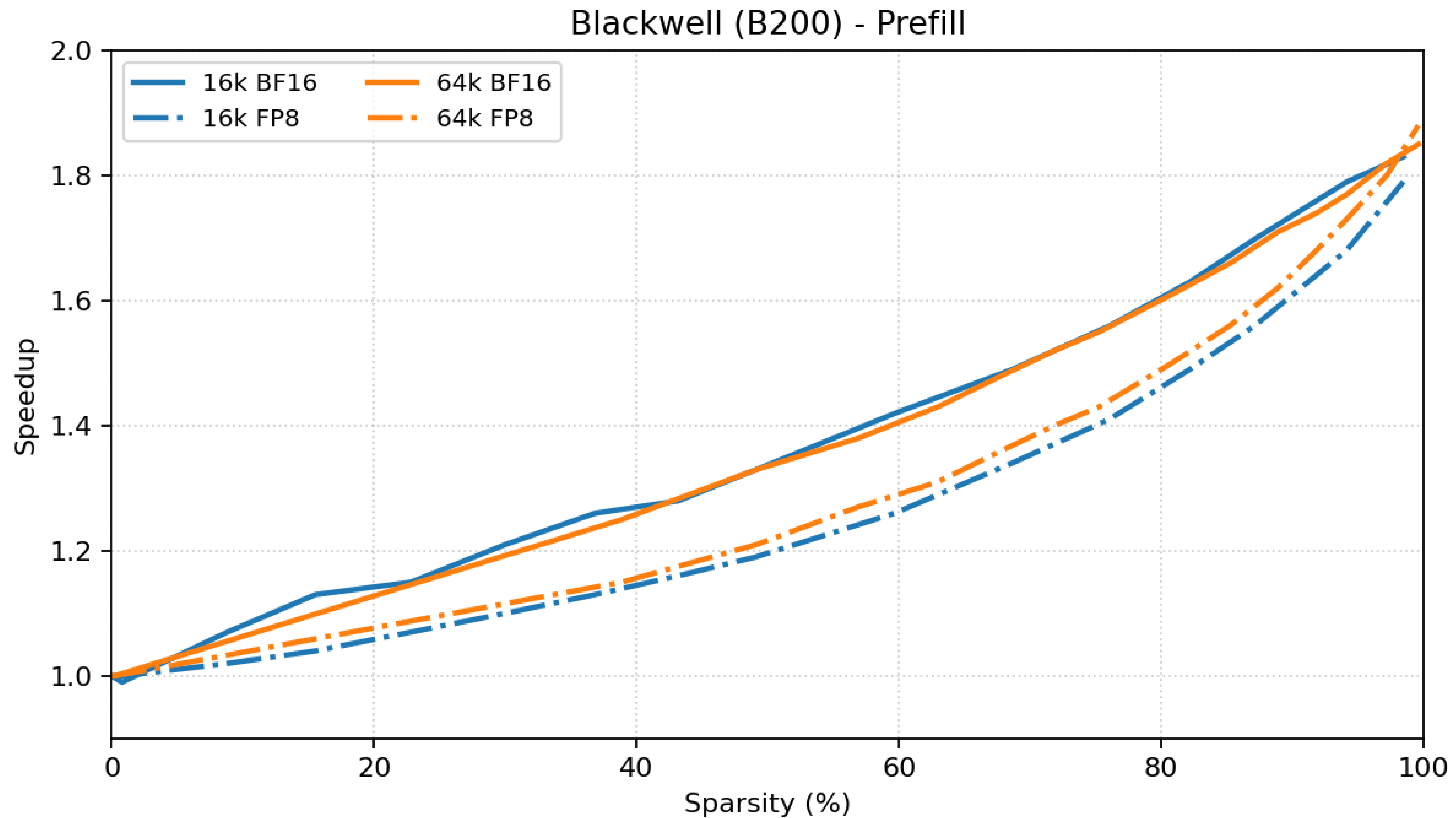


Figure 1. Overview of BLASST. Blocks along a row of the attention matrix are sequentially processed. We (1) update the running row max ($m^{(j)}$) as in FlashAttention, (2) compute the block max ($\tilde{m}^{(j)}$) for each S_j block ($QK_j^\top$), and (3) skip subsequent work if the block max is lower than the running max by more than the input threshold, $\ln(\lambda)$. Full details can be found in Algorithm 1.

Method Properties

- **Ease-of-use**
 - Acceleration fully inside attention kernel
 - No runtime modification – easy to integrate
- **Generality**
 - No training, no fine-tuning
 - Lossy post-training optimization for long-context attention
 - In contrast to VSA
 - Works OOTB for all DiTs, but attention speedup capped at 2x
- Blackwell trtllm-gen kernel support available via **FlashInfer**

Kernel-level Speedup



Intended APIs

TensorRT LLM Reference Integration

- Config via standard LLM API or YAML
- Attention kernel accepts **threshold_scale_factor**
 - In kernel, **threshold** = **threshold_scale_factor** / **seqlen**
 - Legacy design for seqlen adation in LLM
- Or specify **target_sparsity** directly
 - Requires checkpoint calibration (by NVIDIA ModelOpt)
 - Gives a config file telling the relationship between **threshold_scale_factor** and **target_sparsity**
 - No weight modification
- **disabled_until_timestep** at initial denoising steps
 - (Normalized) denoising timestep 1->0
 - Crucial for preserving visual fidelity

<https://github.com/NVIDIA/TensorRT-LLM/blob/main/docs/source/visual-gen/features/sparse-attention.md>

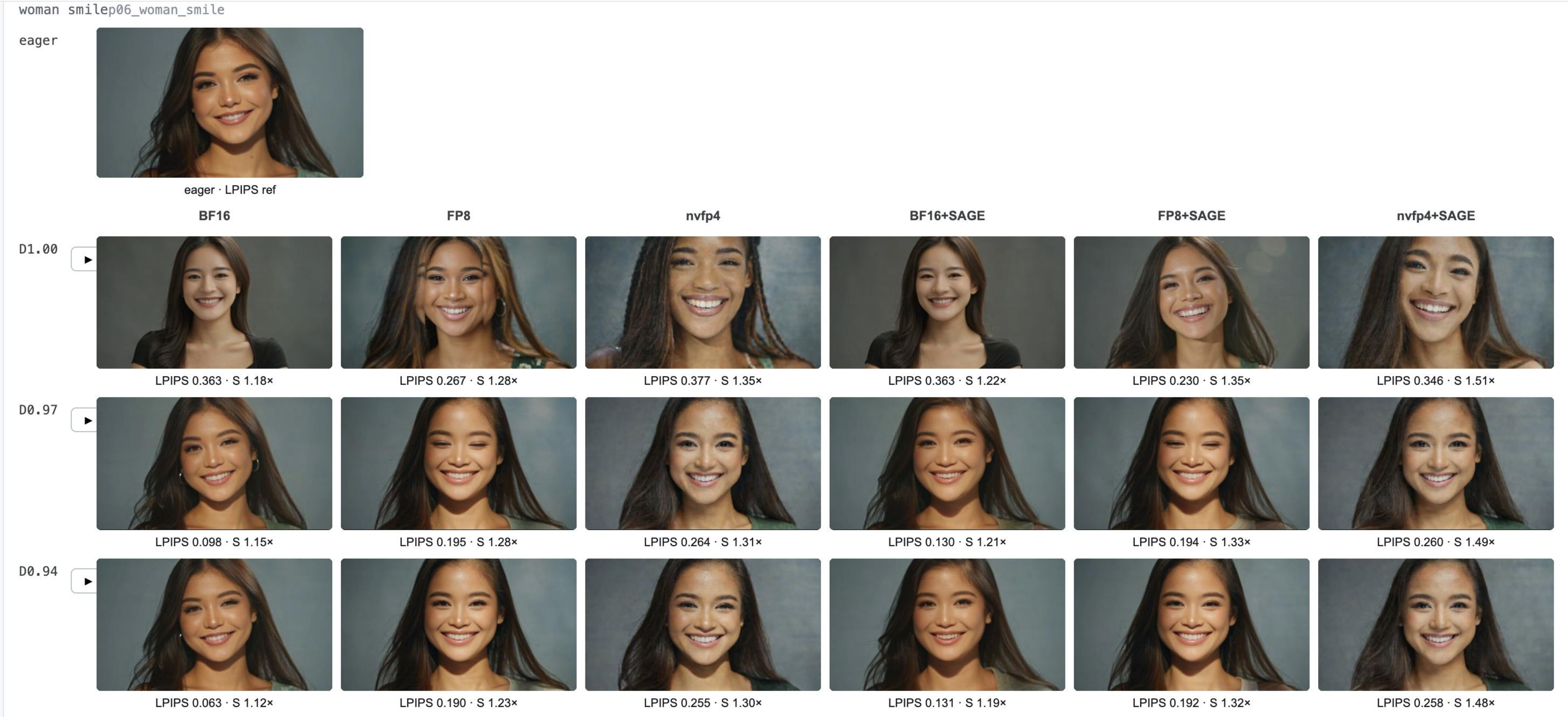
YAML

```
# Direct threshold:
attention_config:
  backend: TRTLLM
  sparse_attention_config:
    algorithm: skip_softmax
    threshold_scale_factor: 5000.0
```

```
# Target sparsity (requires a calibrated checkpoint):
attention_config:
  backend: TRTLLM
  sparse_attention_config:
    algorithm: skip_softmax
    target_sparsity: 0.5
    disabled_until_timestep: 0.6
```

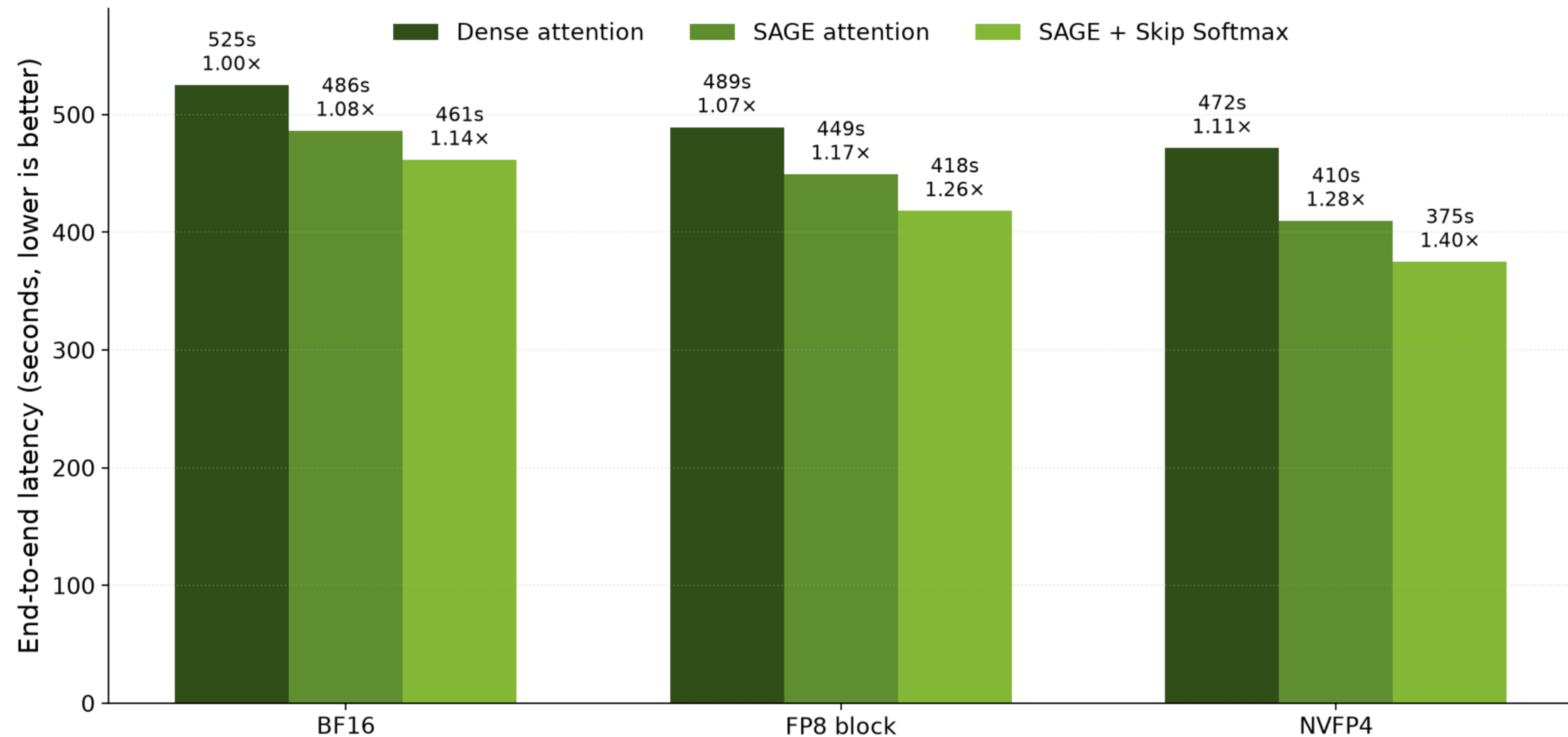

Visual Results

WAN 2.2 A14B, 720×1280 · 81 frames · 50 steps



Experiment Results

Composing GEMM Quant/Sage Attention/Skip Softmax Attention



Wan 2.2 T2V-A14B · 1280×720 · 81 frames · 50 steps · 1× B200 · Skip Softmax: target_sparsity=0.65, disabled_until_timestep=0.86

Experiment Results

E2E Speedup-Fidelity Pareto

